LLM Notes

Module 1: Introduction to Large Language Models

1. Fundamentals

Introduction

Large Language Models (LLMs) are advanced Artificial Intelligence (AI) models designed to **understand, generate, summarize, translate, and interact using human language**.

They learn language by training on massive amounts of text collected from books, articles, websites, research papers, and other publicly available sources.

Unlike traditional programs that follow fixed rules, LLMs learn **patterns, grammar, facts, reasoning, and context** directly from data.

Examples:

- ChatGPT
- Gemini
- Claude
- Llama
- Mistral
- DeepSeek

---

2. What is a Language Model?

A **Language Model (LM)** is an AI model that predicts the **next word or token** in a sequence of text.

Definition

A Language Model estimates the probability of the next word based on the previous words.

Example

Input:

I like to drink

Prediction:

coffee

Sentence becomes:

I like to drink coffee.

Main Goal

Learn patterns of human language so it can

- Predict text
- Complete sentences
- Generate paragraphs
- Answer questions
- Translate languages

Simple Formula

Previous Words → Predict Next Word

---

3. What is a Large Language Model (LLM)?

A **Large Language Model (LLM)** is a language model trained on **billions or trillions of words** using deep learning and the Transformer architecture.

The word **Large** refers to:

- Huge training datasets
- Billions of parameters
- Massive computational resources

Definition

An LLM is a Transformer-based deep learning model capable of understanding and generating human-like text.

Features

- Understands context

- Generates coherent text
- Answers questions
- Writes code
- Summarizes documents
- Translates languages
- Performs reasoning tasks

Examples

- GPT-4
- GPT-5
- Llama
- Claude
- Gemini
- Mistral
- DeepSeek

---

4. Evolution of NLP

Natural Language Processing (NLP) has evolved through four major stages.

Rule-Based NLP
↓
Statistical NLP
↓
Neural NLP
↓
Transformer Era (LLMs)

---

5. Rule-Based NLP

The earliest NLP systems worked using **manually written grammar and language rules**.

Characteristics

- Human-written rules
- No learning from data
- Easy for simple tasks
- Difficult to scale

Example

Rule:

IF input contains "hello"
THEN output "Hi!"

Advantages

- Simple
- Predictable
- Easy to understand

Disadvantages

- Cannot learn
- Poor flexibility
- Doesn't understand context

---

6. Statistical NLP

Instead of handwritten rules, Statistical NLP learns from **large text datasets** using probabilities.

Characteristics

- Data-driven
- Uses probability
- Learns common word patterns

Example

I drink → water (80%)
I drink → tea (15%)
I drink → laptop (0%)

The model predicts the most probable next word.

Advantages

- Better than rule-based systems
- Learns from data
- More flexible

Disadvantages

- Limited context understanding

- Requires feature engineering

---

7. Neural NLP

Neural NLP introduced **Deep Learning** to language processing.

Models automatically learn language features without manual engineering.

Common Models

- RNN
- LSTM
- GRU

Advantages

- Better context understanding
- Learns complex language patterns
- Improved accuracy

Disadvantages

- Slow training
- Difficulty with very long sequences

---

8. Transformer Era

The introduction of the **Transformer architecture (2017)** revolutionized NLP.

Transformers process all words in parallel and use the **Attention Mechanism** to focus on important words.

This led to modern LLMs.

Advantages

- Faster training
- Handles long context
- Better accuracy
- Highly scalable

Popular Models

- GPT
- BERT
- T5
- Llama
- Claude
- Gemini

---

9. Generative AI vs LLM

Generative AI	LLM
Broad field of AI that creates new content	A type of Generative AI focused on text
Generates text, images, audio, video, code	Mainly generates and understands text
Includes many model types	Uses Transformer architecture
Covers multiple modalities	Primarily language-based

Simple Relationship

Generative AI
- LLMs
- Image Models
- Video Models
- Audio Models

---

10. LLM vs Traditional ML Models

Traditional ML	LLM
Task-specific	General-purpose
Requires labeled data	Pretrained on massive text
Needs feature engineering	Learns features automatically
Smaller models	Very large models
Limited language understanding	Strong language understanding

---

11. LLM vs Deep Learning Models

Deep Learning Models	LLM
General neural networks	Transformer-based neural networks
Can work on images, speech, or text	Specialized for language tasks

Deep Learning Models	LLM
Often trained for one task	Can perform many tasks after pretraining
Usually smaller	Typically much larger

Key Point: Every LLM is a deep learning model, but not every deep learning model is an LLM.

---

12. LLM Applications

LLMs are used in many real-world applications, including:

- Chatbots and virtual assistants
- Question answering
- Content writing
- Code generation
- Text summarization
- Language translation
- Grammar correction
- Sentiment analysis
- Document analysis
- Education and tutoring
- Customer support
- Research assistance

---

13. Limitations of LLMs

Despite their capabilities, LLMs have several limitations:

- Can generate incorrect or fabricated information (hallucinations)
- Limited knowledge after their training cutoff unless connected to external data
- May produce biased outputs
- Require large computational resources
- Lack true understanding or consciousness
- Can struggle with complex reasoning in some cases
- Sensitive to prompt wording

---

14. Future of LLMs

Future LLMs are expected to become:

- More accurate and reliable
- Better at reasoning and planning
- Faster and more efficient
- More multimodal (text, image, audio, video)
- More personalized
- Better at using external tools and real-time information
- Smaller enough to run efficiently on personal devices
- More trustworthy with improved safety and factual accuracy

Module 2: Language Modeling Fundamentals

1. Language Modeling

Introduction

Language Modeling is the foundation of Large Language Models (LLMs). It enables a model to understand language patterns and predict the most likely next word or token in a sequence.

A language model learns these patterns by training on large amounts of text.

---

2. What is Language Modeling?

Definition

Language Modeling is the task of predicting the probability of a sequence of words or the next word/token based on previous context.

In simple terms, it helps computers understand **what word is most likely to come next**.

Example

Input:

The sun rises in the

Prediction:

east

Main Goal

- Predict the next word/token
- Understand sentence structure

- Learn grammar and context
- Generate meaningful text

---

3. Probability of Sentences

A language model assigns a **probability (likelihood)** to an entire sentence.

- Higher probability → More natural sentence
- Lower probability → Less natural sentence

Example

 High Probability

I am going to school.

 Low Probability

School going I to am.

The first sentence is more likely because it follows natural language patterns.

---

4. Next Word Prediction

Next Word Prediction is the process of predicting the **most probable next word** using previous words as context.

Example

Input

I love playing

Possible Predictions

- football
- cricket
- games

The model chooses the word with the highest probability.

Why is it Important?

Almost every LLM works by repeatedly predicting the next word or token until a complete response is generated.

---

5. Token Prediction

LLMs do not directly predict words—they predict **tokens**.

A **token** is the smallest unit processed by the model. A token can be:

- A whole word
- Part of a word
- A punctuation mark
- A number
- A special symbol

Example

Sentence

Artificial Intelligence is amazing.

Possible Tokens

Artificial
Intelligence
is
amazing
.

Or

Art
ificial
Intelligence
is
amazing
.

The exact tokenization depends on the tokenizer.

---

6. Conditional Probability

Conditional Probability means finding the probability of an event **given that another event has already occurred**.

In language modeling, the next word depends on the previous words.

Example

Input

I drink

Possible Probabilities

- water → 70%
- tea → 20%
- laptop → 0%

The prediction depends on the given context.

---

7. Chain Rule

The **Chain Rule** breaks the probability of an entire sentence into a sequence of conditional probabilities.

Instead of predicting a full sentence at once, the model predicts **one token at a time** using previous tokens.

Example

Sentence

I love AI

Prediction Order

I
↓
love
↓
AI

Each new word is predicted based on all previous words.

---

8. Context Window

The **Context Window** is the amount of previous text an LLM can remember while generating a response.

It determines how much information the model can use at one time.

Example

Context

John bought a new car yesterday.
Today he drove it to work.

The model understands that **"he"** refers to **John** because it is inside the context window.

Larger Context Window

- Remembers longer conversations
- Better document understanding
- Better reasoning over long text

---

9. Vocabulary

A **Vocabulary** is the complete set of tokens that a language model knows.

Every token has a unique ID used during training and inference.

Example

Vocabulary

cat
dog
apple
computer
AI

The model can only process text after converting it into vocabulary tokens.

---

10. Open Vocabulary

An **Open Vocabulary** model can understand or generate words that were not explicitly seen during training by breaking them into smaller subword tokens.

Example

Unknown Word

ChatGPTification

Possible Tokens

Chat
GPT
ification

This allows modern LLMs to handle new words, names, and technical terms.

Advantages

- Handles new words
- Better multilingual support
- Smaller vocabulary size

---

11. Closed Vocabulary

A **Closed Vocabulary** model only recognizes words present in its predefined vocabulary.

Unknown words cannot be processed directly.

Example

Vocabulary

apple
banana
cat
dog

Input

ChatGPT

Result

Unknown Word (OOV)

(OOV = Out Of Vocabulary)

Disadvantages

- Cannot handle new words
- Limited flexibility
- Common in older language models

---

12. Language Model Objective

The primary objective of a language model is to **predict the next token as accurately as possible**.

By repeating this process, it learns:

- Grammar
- Sentence structure
- Meaning
- Context
- Language patterns

Objective

Previous Tokens
↓
Predict Next Token
↓
Repeat
↓
Generate Complete Text

---

13. Types of Language Models

Language models have evolved over time.

Statistical Language Models
↓
Neural Language Models
↓
Feed Forward Models
↓
Recurrent Models (RNN)
↓
LSTM Models
↓
Transformer Models (Modern LLMs)

---

14. Statistical Language Models

These models use **probability and word frequency** to predict the next word.

Examples

- Unigram
- Bigram
- Trigram
- N-gram

Advantages

- Simple
- Fast
- Easy to understand

Disadvantages

- Cannot capture long context
- Limited accuracy

---

15. Neural Language Models

Neural Language Models use **Artificial Neural Networks (ANNs)** to learn language patterns automatically.

Instead of manually calculating probabilities, they learn from data.

Advantages

- Better context understanding
- Higher accuracy
- Learns semantic relationships

---

16. Feed Forward Language Models

These were the first neural language models.

They use a **Feed Forward Neural Network (FNN)** to predict the next word from a fixed number of previous words.

Example

Input

I love

Prediction

coding

Limitations

- Fixed context size
- Cannot remember long sequences

---

17. Recurrent Language Models (RNN)

Recurrent Neural Networks (RNNs) process text **one word at a time** while carrying information from previous words through a hidden state.

This allows them to remember earlier context better than feed-forward networks.

Advantages

- Handles sequential data
- Better context than FNN

Limitations

- Difficulty remembering very long sequences
- Vanishing gradient problem

---

18. LSTM Language Models

LSTM (**Long Short-Term Memory**) is an improved version of RNN.

It introduces memory cells and gates to remember important information over longer sequences.

Advantages

- Handles long-term dependencies
- Better than RNN
- Reduces vanishing gradient problem

Disadvantages

- Slower training
- More computationally expensive than RNN

---

19. Transformer Language Models

Transformer models are the foundation of modern LLMs.

They use the **Attention Mechanism** to understand relationships between all tokens in a sentence simultaneously instead of processing one word at a time.

Examples

- GPT
- BERT
- Llama
- Gemini
- Claude
- Mistral

Advantages

- Understands long context
- Parallel processing
- Faster training
- High accuracy
- Scales to billions of parameters

Module 3: Text Processing for LLMs

1. Text Processing

Introduction

Before training an LLM, raw text must be converted into a clean and structured format that the model can understand. This process is called **Text Processing (or Text Preprocessing)**.

It involves cleaning text, splitting it into tokens, and creating a vocabulary.

---

2. Text Preprocessing

Definition

Text Preprocessing is the process of cleaning and transforming raw text into a format suitable for training a language model.

Purpose

- Improve text quality
- Remove unnecessary information
- Convert text into tokens
- Prepare data for model training

---

3. Corpus

Definition

A **Corpus** is a large collection of text used to train a language model.

A corpus may contain:

- Books
- Articles
- Websites

- Research papers
- Conversations
- Code
- Documentation

Example

Book 1
Book 2
Wikipedia
News Articles
Blogs

Together, these form a training corpus.

---

4. Cleaning Text

Definition

Text cleaning removes unwanted or inconsistent data before training.

Common Cleaning Steps

- Remove extra spaces
- Remove HTML tags
- Remove duplicate text
- Remove invalid characters
- Fix formatting

Example

Before

Hello!!! Welcome

After

Hello! Welcome

---

5. Unicode

Definition

Unicode is a universal character encoding standard that represents characters from almost every language.

It allows computers to store and display multilingual text correctly.

Example

English : Hello
Hindi : नमस्ते
Japanese: こんにちは
Emoji : □

All are represented using Unicode.

---

6. Normalization

Definition

Normalization converts different representations of text into a consistent format.

Example

Before

ＣｈａｔＧＰＴ

After

ChatGPT

Normalization improves consistency during training.

---

7. Lowercasing

Definition

Lowercasing converts all letters to lowercase.

Example

Before

Machine Learning

After

machine learning

Advantage

- Reduces vocabulary size
- Treats "Apple" and "apple" as the same word (when appropriate)

Note: Many modern LLMs preserve case instead of lowercasing because capitalization can carry meaning.

---

8. Removing Noise

Definition

Noise refers to unnecessary or irrelevant text that does not help model learning.

Examples of Noise

- Extra spaces
- HTML tags
- Random symbols
- Corrupted text
- Duplicate characters

Example

Before

Hello!!!!! ######

After

Hello!

---

9. Tokenization

Definition

Tokenization is the process of splitting text into smaller units called **tokens**.

Tokens can be:

- Characters
- Words
- Subwords
- Sentences

Tokenization is one of the most important steps in LLMs.

---

10. Character Tokenization

Definition

Each character becomes one token.

Example

Sentence

CAT

Tokens

C
A
T

Advantages

- Small vocabulary
- Handles unknown words

Disadvantages

- Very long token sequences
- Slower processing

---

11. Word Tokenization

Definition

Each word becomes one token.

Example

Sentence

I love AI

Tokens

I
love
AI

Advantages

- Simple
- Easy to understand

Disadvantages

- Cannot handle unknown words well
- Requires a large vocabulary

---

12. Subword Tokenization

Definition

Words are split into smaller meaningful pieces called **subwords**.

This is the most common tokenization method used by modern LLMs.

Example

Word

unbelievable

Tokens

un
believe
able

Advantages

- Handles unknown words
- Smaller vocabulary
- Better efficiency

---

13. Sentence Tokenization

Definition

Splits text into individual sentences.

Example

Text

I love AI. It is amazing.

Sentences

I love AI.
It is amazing.

Useful for text analysis and preprocessing.

---

14. Tokenization Algorithms

Tokenization algorithms define **how text is split into tokens**.

Popular algorithms include:

- Byte Pair Encoding (BPE)
- WordPiece
- SentencePiece
- Unigram Language Model
- Byte-Level BPE

---

15. Byte Pair Encoding (BPE)

Definition

BPE starts with individual characters and repeatedly merges the most frequently occurring pairs.

Example

Characters

l o w e r

Possible Merges

lo
low
lower

Advantages

- Efficient vocabulary
- Handles rare words
- Widely used in GPT models

---

16. WordPiece

Definition

WordPiece builds subword tokens based on the likelihood of improving language modeling performance.

Example

Word

playing

Tokens

play
##ing

The ## indicates that the token continues a word.

Used In

- BERT
- DistilBERT

---

17. SentencePiece

Definition

SentencePiece tokenizes text without requiring spaces between words.

It treats the input as a sequence of characters and learns subword units directly.

Advantages

- Works well for many languages
- No language-specific preprocessing
- Handles languages without spaces

Used In

- T5
- ALBERT
- LLaMA

---

18. Unigram Language Model

Definition

The Unigram Language Model starts with many possible subwords and removes less useful ones to create an efficient vocabulary.

Unlike BPE, it **removes** tokens instead of merging them.

Advantages

- Flexible tokenization
- Better handling of multiple segmentations
- Produces high-quality vocabularies

---

19. Byte-Level BPE

Definition

Byte-Level BPE works directly on **bytes** instead of Unicode characters.

This allows it to tokenize **any text**, including emojis and unseen characters.

Advantages

- No unknown characters
- Supports multiple languages
- Handles special symbols

Used In

- GPT-2
- RoBERTa

---

20. Vocabulary

Definition

A **Vocabulary** is the complete set of tokens known by a language model.

Each token is assigned a unique integer ID.

Example

Token	ID
I	1
love	2
AI	3

---

21. Vocabulary Construction

Definition

Vocabulary construction is the process of creating the model's vocabulary from the training corpus.

Steps

1. Collect text corpus
2. Tokenize the text
3. Count token frequencies
4. Select the most useful tokens
5. Assign each token a unique ID

---

22. Vocabulary Size

Definition

Vocabulary size is the total number of unique tokens in the vocabulary.

Example

Model	Approximate Vocabulary Size
BERT	~30,000
GPT-2	~50,000
LLaMA	~32,000

Trade-off

- Small vocabulary → Longer token sequences
- Large vocabulary → More memory required

---

23. Unknown Tokens (UNK)

Definition

An **Unknown Token (UNK)** represents a word that is not present in the model's vocabulary.

Example

Vocabulary

cat
dog
apple

Input

ChatGPT

Output

<UNK>

Modern LLMs rarely use UNK because subword tokenization can usually split unseen words into known subword tokens.

---

24. Special Tokens

Definition

Special tokens are predefined tokens with specific meanings used during training and inference.

They are **not regular words** but help the model understand sentence boundaries, padding, masking, and other tasks.

---

25. PAD Token (<PAD>)

Purpose

Used to make all input sequences the same length by adding empty tokens.

Example

I love AI <PAD> <PAD>

---

26. UNK Token (<UNK>)

Purpose

Represents an unknown or unseen token.

Example

ChatGPT → <UNK>

(Used mainly in older tokenization methods.)

---

27. CLS Token (<CLS>)

Purpose

Added at the **beginning** of a sequence to represent the entire sentence.

Mainly used in **BERT** for classification tasks.

Example

<CLS> I love AI

---

28. SEP Token (<SEP>)

Purpose

Separates two sentences or marks the end of a sentence.

Example

Sentence 1 <SEP> Sentence 2

---

29. MASK Token (<MASK>)

Purpose

Replaces hidden words during training so the model learns to predict them.

Example

I love <MASK>.

Prediction

AI

Mainly used in **BERT**.

---

30. BOS Token (<BOS>)

Purpose

Marks the **Beginning Of Sequence**.

Example

<BOS> Hello everyone.

---

31. EOS Token (<EOS>)

Purpose

Marks the **End Of Sequence**.

It tells the model when text generation should stop.

Example

Hello everyone. <EOS>

Module 4: Word Representation

1. Embeddings

Introduction

Computers cannot understand words directly. They only understand numbers.

Embeddings convert words into numerical vectors while preserving their meaning and relationships.

---

2. One-Hot Encoding

Definition

One-Hot Encoding represents each word as a binary vector where only one position is **1** and all others are **0**.

Example

Word	Vector
Cat	[1,0,0]
Dog	[0,1,0]
Bird	[0,0,1]

Advantages

- Simple
- Easy to implement

Disadvantages

- Very large vectors
- No semantic meaning
- Similar words appear unrelated

---

3. Word Embeddings

Definition

Word Embeddings represent words as **dense numerical vectors** where similar words have similar vector values.

Example

King → [0.32, 0.81, -0.25, ...]
Queen → [0.30, 0.79, -0.22, ...]

King and Queen have similar meanings, so their vectors are close.

---

4. Dense Representation

Definition

A **Dense Representation** is a vector where most values are non-zero.

Example

[0.25, -0.64, 0.91, 0.13]

Advantages

- Compact
- Captures semantic meaning
- More efficient than one-hot encoding

---

5. Static Embeddings

Definition

Static embeddings assign **one fixed vector** to a word regardless of context.

Example

The word **"bank"** has the same embedding in:

- River bank
- Bank account

Examples

- Word2Vec
- GloVe
- FastText

---

6. Contextual Embeddings

Definition

Contextual embeddings generate **different vectors** for the same word depending on its context.

Example

- River **bank**
- Money **bank**

The embedding changes based on the sentence.

Examples

- BERT
- GPT
- LLaMA
- Gemini

---

7. Embedding Models

Embedding models learn meaningful vector representations of words.

Popular embedding models:

- Word2Vec
- GloVe
- FastText

---

8. Word2Vec

Definition

Word2Vec is a neural network model that learns word embeddings by predicting surrounding words.

Advantages

- Fast
- Captures semantic relationships
- Produces high-quality embeddings

---

9. CBOW (Continuous Bag of Words)

Definition

CBOW predicts the **current word** using surrounding context words.

Example

I love ___ very much.

Prediction:

AI

Characteristics

- Faster training
- Works well for frequent words

---

10. Skip-Gram

Definition

Skip-Gram predicts the **surrounding words** using the current word.

Example

Input

AI

Predictions

love
future
technology

Characteristics

- Better for rare words
- Slightly slower than CBOW

---

11. GloVe

Definition

GloVe (**Global Vectors**) learns embeddings using **global word co-occurrence statistics**.

Instead of only nearby words, it considers how often words appear together across the entire corpus.

Advantages

- Captures global relationships
- High-quality embeddings

---

12. FastText

Definition

FastText represents words using **subword (character n-gram)** information.

Example

Word

playing

Subwords

play
lay
aying
ing

Advantages

- Handles rare words
- Handles unknown words
- Better for morphologically rich languages

---

13. Embedding Matrix

Definition

An **Embedding Matrix** is a table where each row represents the embedding vector of one token.

Example

Word	Vector
Cat	[0.2, 0.5, -0.3]
Dog	[0.1, 0.7, -0.4]
AI	[0.9, 0.3, 0.6]

---

14. Similarity

Definition

Similarity measures how close two word embeddings are in meaning.

Example

High Similarity

Car ↔ Vehicle

Low Similarity

Car ↔ Banana

---

15. Cosine Similarity

Definition

Cosine Similarity measures the similarity between two embedding vectors by comparing the angle between them.

Value Range

- **1** → Exactly similar
- **0** → Unrelated
- **-1** → Opposite direction

Example

King ↔ Queen → High similarity
Apple ↔ Car → Low similarity

---

16. Semantic Relationships

Definition

Embeddings capture relationships between words based on meaning.

Examples

- King ↔ Queen
- Paris ↔ France
- Doctor ↔ Hospital

These words have similar meanings or related concepts, so their vectors are close.

---

17. Embedding Dimension

Definition

Embedding Dimension is the number of values in each embedding vector.

Example

[0.25, -0.64, 0.81, 0.42]

Embedding Dimension = **4**

Modern LLMs often use embedding sizes ranging from hundreds to several thousand dimensions.

Module 5: Transformer Foundations

1. Why Transformers?

Introduction

Before Transformers, language models mainly used **RNNs** and **LSTMs**. These models struggled with long texts and were slow to train.

Transformers solved these problems using the **Attention Mechanism**, making modern LLMs possible.

---

2. Limitations of RNN

Definition

RNNs process words **one at a time** in sequence.

Limitations

- Slow training
- Cannot process words in parallel
- Difficulty remembering long context
- Vanishing gradient problem

---

3. Limitations of LSTM

Definition

LSTMs improved RNNs by adding memory cells but still process text sequentially.

Limitations

- Sequential computation
- Slower training
- High computational cost
- Limited scalability for very large models

---

4. Long-Term Dependencies

Definition

Long-term dependency means connecting words that are far apart in a sentence.

Example

The book that I borrowed from my friend yesterday was interesting.

The model should connect **book** and **interesting**, even though many words separate them.

Transformers handle long-term dependencies much better than RNNs and LSTMs.

---

5. Parallel Processing

Definition

Transformers process **all tokens simultaneously** instead of one after another.

Advantages

- Faster training
- Better GPU utilization
- Scales to very large datasets

---

6. Transformer Overview

Definition

A **Transformer** is a deep learning architecture based on the **Attention Mechanism**.

It understands relationships between all words in a sequence and forms the foundation of modern LLMs.

---

7. Encoder

Definition

The **Encoder** reads the input text and converts it into meaningful contextual representations.

Used In

- BERT
- RoBERTa

Best For

- Text understanding
- Classification
- Question answering

---

8. Decoder

Definition

The **Decoder** generates text one token at a time using previously generated tokens.

Used In

- GPT
- LLaMA
- Gemini (decoder component)

Best For

- Text generation
- Chatbots
- Code generation

---

9. Encoder-Decoder Architecture

Definition

Uses both an encoder and a decoder.

- Encoder understands the input.
- Decoder generates the output.

Examples

- T5
- BART

Applications

- Translation
- Summarization
- Text-to-text tasks

---

10. Decoder-Only Architecture

Definition

Contains only a decoder and predicts the next token.

Examples

- GPT
- LLaMA
- Mistral

Applications

- Chatbots
- Text generation
- Coding assistants

---

11. Encoder-Only Architecture

Definition

Contains only an encoder and focuses on understanding text rather than generating it.

Examples

- BERT
- RoBERTa

Applications

- Sentiment analysis
- Text classification

- Named Entity Recognition (NER)

---

12. Core Components

The main building blocks of a Transformer are:

- Self-Attention
- Multi-Head Attention
- Positional Encoding
- Feed Forward Network
- Residual Connections
- Layer Normalization
- Dropout

---

13. Self-Attention

Definition

Self-Attention allows each token to focus on other important tokens in the same sentence.

Example

The cat drank its milk because it was hungry.

The model understands that **it** refers to **the cat**, not **milk**.

Advantages

- Captures context
- Handles long-distance relationships
- Improves understanding

---

14. Multi-Head Attention

Definition

Instead of using one attention mechanism, the Transformer uses **multiple attention heads**.

Each head learns different relationships in the sentence.

Advantage

Different heads can focus on:

- Grammar
- Meaning
- Word order
- Long-distance dependencies

---

15. Positional Encoding

Definition

Transformers process tokens in parallel, so they need positional information to know the order of words.

Positional Encoding adds this order information to token embeddings.

Example

I love AI

The model knows:

- I → Position 1
- love → Position 2
- AI → Position 3

---

16. Feed Forward Network (FFN)

Definition

After attention, each token passes through a small neural network called the **Feed Forward Network**.

Purpose

- Learns more complex patterns
- Refines token representations
- Increases model capacity

---

17. Residual Connections

Definition

Residual connections add the input of a layer directly to its output.

Benefits

- Easier training
- Better information flow
- Reduces gradient problems in deep networks

---

18. Layer Normalization

Definition

Layer Normalization normalizes activations within a layer to stabilize training.

Benefits

- Faster convergence
- Stable learning
- Better overall performance

---

19. Dropout

Definition

Dropout randomly disables some neurons during training to reduce overfitting.

Benefits

- Prevents overfitting
- Improves generalization
- Makes the model more robust

Module 6: Attention Mechanism

1. Attention

Introduction

The **Attention Mechanism** is the core idea behind Transformers and modern LLMs.

It allows the model to **focus on the most relevant words (tokens)** in a sentence while processing each token.

Instead of treating every word equally, attention assigns **higher importance to relevant words**.

---

2. Why Attention?

Before attention, models like RNNs and LSTMs struggled to remember information from long sentences.

Attention solves this by allowing every token to directly look at other important tokens.

Benefits

- Understands long context
- Captures relationships between distant words
- Improves translation and text generation
- Enables parallel processing

---

3. Attention Intuition

Intuition

When humans read a sentence, they naturally focus on important words.

Similarly, attention lets the model decide **which words deserve more focus**.

Example

Sentence

The cat sat on the mat because it was tired.

To understand **"it"**, the model pays more attention to **"cat"** than to **"mat"**.

---

4. Query (Q)

Definition

A **Query (Q)** represents **what the current token is looking for**.

It is used to search for relevant information from other tokens.

Example

If the token is **"it"**, its Query asks:

"Which previous word does 'it' refer to?"

---

5. Key (K)

Definition

A **Key (K)** represents **what information each token contains**.

The Query is compared with all Keys to find the most relevant tokens.

Example

Sentence

The cat drank milk because it was hungry.

The Key of **"cat"** matches well with the Query of **"it"**.

---

6. Value (V)

Definition

A **Value (V)** contains the **actual information** that will be used after attention identifies the relevant tokens.

Simple Idea

- Query → What am I looking for?
- Key → What information do I have?
- Value → The information returned

---

7. Attention Scores

Definition

Attention Scores measure **how relevant one token is to another**.

Higher score → More attention

Lower score → Less attention

Example

Sentence

The cat chased the mouse because it was fast.

Possible Scores

it → cat = High
it → mouse = Medium
it → the = Low

The model focuses more on tokens with higher scores.

---

8. Scaled Dot Product Attention

Definition

Scaled Dot Product Attention is the **main attention calculation** used in Transformers.

It compares **Queries** and **Keys** to compute attention scores, scales the scores, applies Softmax, and then combines the **Values**.

Steps

1. Compare Query and Key
2. Calculate attention scores
3. Scale the scores
4. Apply Softmax
5. Weight the Values
6. Produce the attention output

Why Scaling?

Scaling prevents very large attention scores, making training more stable.

---

9. Softmax

Definition

Softmax converts attention scores into probabilities.

The probabilities always sum to **1**.

Example

Raw Scores

2.0
1.0
0.5

After Softmax

0.63
0.23
0.14

The model gives the highest attention to the first token.

---

10. Multi-Head Attention

Definition

Instead of using one attention mechanism, Transformers use **multiple attention heads**.

Each head learns different relationships in the sentence.

Example

Different heads may focus on:

- Grammar
- Meaning
- Subject-object relationship
- Long-distance dependencies

Advantages

- Learns multiple patterns simultaneously
- Better language understanding
- Improves model performance

---

11. Types of Attention

Common attention mechanisms include:

- Self Attention
- Cross Attention
- Masked Attention
- Causal Attention
- Bidirectional Attention
- Sparse Attention
- Sliding Window Attention

---

12. Self Attention

Definition

In **Self Attention**, every token attends to **other tokens within the same sequence**.

Example

The dog wagged its tail.

The token **"its"** attends to **"dog"** to understand its meaning.

Used In

- BERT
- GPT
- LLaMA
- Most Transformer models

---

13. Cross Attention

Definition

In **Cross Attention**, tokens from one sequence attend to tokens in **another sequence**.

Example

Translation

English Input
↓

French Output

The decoder attends to the encoder output while generating the translation.

Used In

- Encoder-Decoder models
- T5
- BART

---

14. Masked Attention

Definition

Masked Attention hides certain future tokens so they cannot influence the current prediction.

This prevents the model from "seeing the future."

Example

While predicting:

I love ____

The model cannot look ahead at the correct answer.

Purpose

- Prevents information leakage
- Enables autoregressive text generation

---

15. Causal Attention

Definition

Causal Attention allows a token to attend **only to previous tokens and itself**, never to future tokens.

Example

Current Token

Today

Can attend to

I
will
travel
Today

Cannot attend to

tomorrow

Used In

- GPT
- LLaMA
- Mistral

---

16. Bidirectional Attention

Definition

Bidirectional Attention allows each token to attend to **both previous and future tokens**.

Example

The movie was surprisingly good.

The word **"surprisingly"** can attend to both **"movie"** and **"good"**.

Used In

- BERT
- RoBERTa

Best For

- Text understanding
- Classification
- Question answering

---

17. Sparse Attention

Definition

Sparse Attention allows a token to attend to **only a subset of tokens**, rather than every token.

Advantages

- Lower memory usage
- Faster computation
- Better for very long documents

Used In

Long-context Transformer models.

---

18. Sliding Window Attention

Definition

Sliding Window Attention allows each token to attend only to **nearby tokens within a fixed window**.

Example

Window Size = 3

A B C D E F G

Token **D** attends to:

B C D E F

but not to **A** or **G**.

Advantages

- Efficient for long sequences

- Lower computational cost
- Suitable for long-document processing

Module 7: Positional Information

1. Positional Information

Introduction

Transformers process **all tokens at the same time (parallel processing)**. Unlike RNNs, they do not naturally know the order of words.

Positional Information tells the model where each token appears in a sequence.

---

2. Positional Encoding

Definition

Positional Encoding (PE) adds position information to token embeddings so the Transformer understands word order.

Example

Sentence

I love AI

Positions

I → Position 1
love → Position 2
AI → Position 3

The model combines the token embedding with its positional information.

---

3. Need for Position Encoding

Without positional information, these sentences would look identical to the Transformer:

Dog bites man.
Man bites dog.

Although the words are the same, the meanings are completely different.

Why is it Needed?

- Preserves word order
- Helps understand sentence meaning
- Improves language understanding
- Enables correct context interpretation

---

4. Absolute Positional Encoding

Definition

Each position in a sequence is assigned a **fixed position value**.

The position depends only on the token's location.

Example

Position 1 → I
Position 2 → love
Position 3 → AI

Each position has its own unique encoding.

---

5. Sinusoidal Encoding

Definition

Sinusoidal Encoding generates positional values using **sine and cosine functions** instead of learning them.

It was introduced in the original Transformer paper.

Advantages

- No extra parameters to learn
- Works with longer sequences
- Generalizes to unseen positions

Used In

Original Transformer (Attention Is All You Need)

---

6. Learned Positional Embeddings

Definition

Instead of fixed values, the model **learns positional embeddings during training**.

Each position has a trainable vector.

Advantages

- Learns task-specific position information
- Often performs better for fixed sequence lengths

Limitation

May not generalize well to much longer sequences than seen during training.

---

7. Relative Position Encoding

Definition

Relative Position Encoding focuses on the **distance between tokens** rather than their absolute positions.

Example

Sentence

The cat chased the mouse.

Instead of using:

cat = Position 2
mouse = Position 5

The model learns that **mouse is 3 positions away from cat**.

Advantages

- Better captures relationships between nearby tokens

- Works well for varying sequence lengths

---

8. Rotary Positional Embedding (RoPE)

Definition

RoPE (Rotary Positional Embedding) encodes position by applying rotational transformations to token embeddings.

It naturally captures relative position information while preserving word relationships.

Advantages

- Excellent for long context
- Better extrapolation to longer sequences
- Efficient and widely used in modern LLMs

Used In

- LLaMA
- GPT-NeoX
- Qwen
- Mistral

---

9. ALiBi (Attention with Linear Biases)

Definition

ALiBi adds a linear bias to attention scores based on the distance between tokens instead of using explicit positional embeddings.

Advantages

- Supports very long contexts
- Simple implementation
- No learned positional embeddings required
- Better length generalization

Used In

Several long-context Transformer models.

Module 8: Transformer Architectures

1. Architectures

Introduction

Transformer architectures define **how the encoder and decoder are arranged** to perform different NLP tasks.

There are three main Transformer architectures:

- Encoder-Only
- Decoder-Only
- Encoder-Decoder

---

2. Encoder-Only Models

Definition

Encoder-only models use **only the Transformer encoder**.

Their primary purpose is **understanding text**, not generating it.

Characteristics

- Reads the entire input at once
- Uses bidirectional attention
- Produces contextual representations

Best For

- Text classification
- Sentiment analysis
- Named Entity Recognition (NER)
- Question answering

Example

- BERT

---

3. Decoder-Only Models

Definition

Decoder-only models use **only the Transformer decoder**.

They generate text by predicting the **next token** one token at a time.

Characteristics

- Uses causal (masked) attention
- Generates text autoregressively
- Excellent for text generation

Best For

- Chatbots
- Content generation
- Code generation
- Story writing

Examples

- GPT
- LLaMA
- Mistral

---

4. Encoder-Decoder Models

Definition

Encoder-decoder models contain **both an encoder and a decoder**.

- Encoder understands the input.
- Decoder generates the output.

Best For

- Machine translation
- Text summarization
- Text-to-text tasks

Examples

- T5
- BART
- UL2

---

5. Popular Architectures

Some of the most influential Transformer architectures are:

- BERT
- GPT
- T5
- BART
- UL2

---

6. BERT

Full Form

Bidirectional Encoder Representations from Transformers

Architecture

- Encoder-Only

Characteristics

- Uses bidirectional attention
- Reads both left and right context
- Strong language understanding

Applications

- Text classification
- Question answering
- Sentiment analysis
- NER

---

7. GPT

Full Form

Generative Pre-trained Transformer

Architecture

- Decoder-Only

Characteristics

- Uses causal attention
- Predicts the next token
- Optimized for text generation

Applications

- Chatbots
- Writing assistants
- Coding assistants
- Content generation

---

8. T5

Full Form

Text-to-Text Transfer Transformer

Architecture

- Encoder-Decoder

Key Idea

Every NLP task is converted into a **text-to-text** problem.

Example

Input

Translate English to French:
Hello

Output

Bonjour

Applications

- Translation
- Summarization
- Question answering
- Text generation

---

9. BART

Full Form

Bidirectional and Auto-Regressive Transformers

Architecture

- Encoder-Decoder

Characteristics

- Encoder understands corrupted input
- Decoder reconstructs the original text

Applications

- Summarization
- Text generation
- Text correction

---

10. UL2

Full Form

Unified Language Learner

Architecture

- Encoder-Decoder

Characteristics

- Combines multiple pretraining objectives
- Handles a wide variety of language tasks

- Improves generalization across tasks

Applications

- General-purpose NLP
- Text generation
- Question answering
- Summarization

---

11. BERT vs GPT

Feature	BERT	GPT
Architecture	Encoder-Only	Decoder-Only
Attention	Bidirectional	Causal (Unidirectional)
Primary Goal	Text Understanding	Text Generation
Predicts	Masked tokens	Next token
Best For	Classification, QA	Chatbots, Writing, Coding

---

12. GPT vs T5

Feature	GPT	T5
Architecture	Decoder-Only	Encoder-Decoder
Input	Text Prompt	Text Input
Output	Next-token generation	Text-to-text generation
Strength	Open-ended generation	Structured NLP tasks
Examples	Chatbots, Code	Translation, Summarization

---

13. Encoder vs Decoder

Encoder	Decoder
Understands input	Generates output
Uses bidirectional attention	Uses causal attention
Reads the entire sequence	Predicts one token at a time
Best for understanding	Best for generation

---

14. Decoder-Only Advantages

Advantages

- Simple architecture
- Excellent text generation
- Scales efficiently to very large models
- Performs well in zero-shot and few-shot learning
- Ideal for conversational AI
- Strong coding and reasoning capabilities

Examples

- GPT
- LLaMA
- Mistral
- Qwen

Module 9: LLM Training

1. LLM Training

Introduction

LLMs learn language by training on **massive amounts of text**. During training, the model repeatedly predicts missing or next tokens, compares its predictions with the correct answers, and updates its parameters to improve.

Training consists of:

- Pretraining
- Learning from large datasets
- Optimizing model parameters

---

2. Pretraining

Definition

Pretraining is the first stage of LLM training where the model learns general language knowledge from a huge text corpus.

It learns:

- Grammar
- Vocabulary
- Facts

- Reasoning patterns
- Language structure

Example

The model reads billions of sentences before being used for specific tasks.

---

3. Self-Supervised Learning

Definition

Self-Supervised Learning is a training method where the **training data itself provides the labels**.

No human annotation is required.

Example

Sentence

I love _____.

Target

AI

The model learns by predicting the missing or next token.

Advantages

- No manual labeling
- Uses large amounts of text
- Scales easily

---

4. Causal Language Modeling (CLM)

Definition

Causal Language Modeling trains the model to **predict the next token** using only previous tokens.

Example

Input

I love

Prediction

AI

Used In

- GPT
- LLaMA
- Mistral

---

5. Masked Language Modeling (MLM)

Definition

Masked Language Modeling hides some words in a sentence, and the model predicts the missing words.

Example

Input

I love <MASK>.

Prediction

AI

Used In

- BERT
- RoBERTa

---

6. Prefix Language Modeling

Definition

Prefix Language Modeling allows the model to see a **given prefix (prompt)** and predict the remaining text.

Example

Prefix

Write a story about a dragon.

Output

The model generates the rest of the story.

Benefit

Useful for instruction-following and text generation.

---

7. Span Corruption

Definition

Instead of masking a single word, **Span Corruption** masks an entire sequence of consecutive words.

Example

Original

The cat is sitting on the mat.

Training Input

The <MASK> the mat.

Target

cat is sitting on

Used In

- T5
- UL2

---

8. Next Token Prediction

Definition

The model predicts the **next token** based on all previous tokens.

Example

Input

Artificial Intelligence is

Prediction

powerful

This is the main learning objective for most modern LLMs.

---

9. Dataset

Definition

A **Dataset** is the collection of text used to train an LLM.

A good dataset should be:

- Large
- Diverse
- High quality
- Well cleaned

---

10. Web Data

Definition

Web data consists of publicly available internet content.

Examples

- Blogs
- News articles
- Documentation
- Forums
- Websites

Advantage

Provides diverse real-world language.

---

11. Books

Definition

Books provide long-form, well-written text.

Benefits

- Rich vocabulary
- Grammar
- Logical structure
- Long-context learning

---

12. Wikipedia

Definition

Wikipedia is a large collection of structured knowledge articles.

Benefits

- High-quality information
- General knowledge
- Multiple languages
- Reliable writing style

---

13. Code

Definition

Programming code is included to teach LLMs software development.

Examples

- Python
- Java

- C++
- JavaScript

Benefit

Enables code generation, debugging, and explanation.

---

14. Academic Papers

Definition

Academic papers provide scientific and technical knowledge.

Benefits

- Research concepts
- Mathematics
- AI
- Medicine
- Engineering

---

15. Data Quality

Definition

Data Quality refers to how clean, accurate, and useful the training data is.

Good Quality Data

- Correct grammar
- Accurate facts
- Diverse topics
- Minimal duplicates
- Safe content

Better data leads to better models.

---

16. Data Deduplication

Definition

Data Deduplication removes repeated or duplicate text from the dataset.

Benefits

- Prevents memorization
- Reduces bias toward repeated content
- Improves training efficiency

---

17. Filtering

Definition

Filtering removes unwanted or low-quality data before training.

Examples of Filtered Content

- Spam
- Offensive content
- Corrupted text
- Very low-quality documents
- Duplicate data

---

18. Training Pipeline

Definition

The **Training Pipeline** is the complete sequence of steps used to train an LLM.

Dataset Collection
↓
Text Cleaning
↓
Tokenization
↓
Batching
↓
Forward Pass
↓
Loss Computation
↓
Backpropagation
↓

Optimization
↓
Checkpoint Saving
↓
Repeat

---

19. Dataset Collection

Definition

The first step is collecting text from multiple sources.

Sources

- Books
- Websites
- Wikipedia
- Code
- Research papers

---

20. Tokenization

Definition

The collected text is converted into **tokens** that the model can process.

Example

I love AI

↓

I | love | AI

---

21. Batching

Definition

Instead of processing one example at a time, training data is divided into **batches**.

Example

Batch 1 → 32 sentences
Batch 2 → 32 sentences
Batch 3 → 32 sentences

Advantages

- Faster training
- Better GPU utilization
- More stable optimization

---

22. Forward Pass

Definition

During the **Forward Pass**, the input tokens pass through the model to produce predictions.

Example

Input

I love

Prediction

AI

---

23. Loss Computation

Definition

Loss measures how far the model's prediction is from the correct answer.

Example

Correct Token

AI

Predicted Token

robot

Higher error → Higher loss.

Goal

Minimize the loss during training.

---

24. Backpropagation

Definition

Backpropagation calculates how each model parameter contributed to the error.

These gradients are then used to update the model.

Purpose

- Learn from mistakes
- Improve future predictions

---

25. Optimization

Definition

Optimization updates the model parameters to reduce the loss.

Popular optimizers include:

- SGD (Stochastic Gradient Descent)
- Adam
- AdamW

Goal

Improve prediction accuracy after every training step.

---

26. Checkpointing

Definition

Checkpointing saves the model periodically during training.

Benefits

- Resume training if interrupted
- Save intermediate models
- Compare different training stages
- Prevent loss of training progress

Module 10: Optimization

1. Training Optimization

Introduction

Training an LLM involves adjusting millions or billions of parameters to reduce prediction errors.

Optimization is the process of updating model parameters so the model becomes more accurate after each training step.

---

2. Gradient Descent

Definition

Gradient Descent is an optimization algorithm that minimizes the model's loss by updating its parameters in the direction that reduces the error.

Simple Idea

Prediction
↓
Calculate Error
↓
Update Parameters
↓
Better Prediction

Goal

Reduce the loss after every update.

---

3. SGD (Stochastic Gradient Descent)

Definition

SGD updates model parameters using a **small batch of training examples** instead of the entire dataset.

Advantages

- Faster updates
- Lower memory usage
- Suitable for large datasets

Limitation

Updates can be noisy and less stable.

---

4. Adam

Definition

Adam (Adaptive Moment Estimation) is one of the most widely used optimizers for deep learning.

It automatically adjusts the learning rate for each parameter.

Advantages

- Fast convergence
- Stable training
- Works well for large models

---

5. AdamW

Definition

AdamW is an improved version of Adam that applies **Weight Decay** separately from gradient updates.

Advantages

- Better generalization
- Reduces overfitting
- Commonly used for Transformer training

---

6. Learning Rate

Definition

The **Learning Rate (LR)** determines **how much model parameters are updated** after each training step.

Effect

- High LR → Faster learning but may become unstable.
- Low LR → Stable learning but slower training.

Choosing the right learning rate is crucial.

---

7. Warmup

Definition

Warmup starts training with a very small learning rate and gradually increases it during the first few training steps.

Benefits

- Stable early training
- Prevents sudden large updates
- Improves convergence

---

8. Learning Rate Scheduling

Definition

A **Learning Rate Scheduler** changes the learning rate during training according to a predefined strategy.

Common Strategies

- Constant
- Step Decay
- Linear Decay
- Cosine Decay

Benefits

- Faster convergence
- Better final performance
- Stable optimization

---

9. Weight Decay

Definition

Weight Decay is a regularization technique that discourages very large parameter values.

Benefits

- Reduces overfitting
- Improves generalization
- Commonly used with AdamW

---

10. Gradient Clipping

Definition

Gradient Clipping limits the maximum size of gradients during training.

Benefits

- Prevents exploding gradients
- Improves training stability
- Useful for very deep models

---

11. Batch Size

Definition

Batch Size is the number of training examples processed together before updating the model.

Example

Batch Size = **32**

The model processes 32 examples, then updates its parameters.

Trade-off

- Small Batch → Lower memory, noisier updates
- Large Batch → Faster training, higher memory usage

---

12. Epoch

Definition

An **Epoch** is one complete pass through the entire training dataset.

Example

Dataset = **10,000 samples**

After processing all 10,000 samples once:

1 Epoch Completed

---

13. Iteration

Definition

An **Iteration** is one batch processed during training.

Example

Dataset = **1,000 samples**

Batch Size = **100**

Iterations per Epoch = **10**

---

14. Step

Definition

A **Step** is one parameter update performed after processing a batch.

Generally:

- One batch → One forward pass
- One backward pass
- One optimization update
- One training step

---

15. Stability

Definition

Training Stability means the model learns smoothly without sudden increases in loss or unstable behavior.

Stable Training Requires

- Proper learning rate
- Good optimizer
- Gradient clipping
- High-quality data

---

16. Gradient Explosion

Definition

Gradient Explosion occurs when gradients become extremely large during backpropagation.

Effects

- Unstable training
- Very large parameter updates
- Training may fail

Solution

- Gradient Clipping
- Smaller learning rate

---

17. Gradient Vanishing

Definition

Gradient Vanishing occurs when gradients become extremely small.

Effects

- Slow learning
- Early layers stop updating
- Poor performance

Solution

- Residual Connections
- Layer Normalization
- Better activation functions
- Transformer architecture

---

18. Mixed Precision

Definition

Mixed Precision Training uses different numerical precisions together to improve speed and reduce memory usage.

Typically combines:

- FP32
- FP16 or BF16

Benefits

- Faster training
- Lower GPU memory usage
- Enables larger models

---

19. FP16

Definition

FP16 (16-bit Floating Point) stores numbers using 16 bits.

Advantages

- Faster computation

- Lower memory usage

Limitation

Lower numerical precision than FP32.

---

20. BF16

Definition

BF16 (Brain Floating Point 16) is another 16-bit format designed for deep learning.

Advantages

- Better numerical stability than FP16
- Lower memory usage than FP32
- Widely supported on modern AI hardware

---

21. Gradient Accumulation

Definition

Gradient Accumulation simulates a larger batch size by accumulating gradients across multiple smaller batches before updating the model.

Example

Batch Size = **8**

Accumulate for **4 batches**

Effective Batch Size = **32**

Benefits

- Trains large models with limited GPU memory
- Improves flexibility

Module 11: Scaling Laws

1. Scaling

Introduction

As LLMs become **larger**, use **more data**, and receive **more computation**, their performance generally improves.

These relationships are described by **Scaling Laws**.

---

2. Model Size

Definition

Model Size refers to the total number of **trainable parameters** in an LLM.

Examples

- Small Model → Millions of parameters
- Medium Model → Billions of parameters
- Large Model → Hundreds of billions of parameters

Effect

Larger models can learn more complex language patterns but require more compute and memory.

---

3. Dataset Size

Definition

Dataset Size is the total amount of training data used during pretraining.

Examples

- Books
- Wikipedia
- Web pages
- Code
- Research papers

Effect

Larger and higher-quality datasets generally improve model performance.

---

4. Compute Budget

Definition

Compute Budget is the total computational resources available for training.

It includes:

- GPUs/TPUs
- Training time
- Memory
- Energy consumption

Effect

More compute allows training larger models on more data, but increases cost.

---

5. Scaling Laws

Definition

Scaling Laws describe how model performance changes as **model size**, **dataset size**, and **compute** increase.

Key Idea

Increasing all three in a balanced way usually leads to better performance.

---

6. Chinchilla Scaling Law

Definition

The **Chinchilla Scaling Law** states that, for a fixed compute budget, it is often better to train a **smaller model on more high-quality data** than a much larger model on less data.

Key Insight

Balance between:

- Model size
- Dataset size
- Compute budget

This leads to more compute-efficient training.

---

7. Emergent Abilities

Definition

Emergent Abilities are new capabilities that appear when an LLM becomes sufficiently large, even though these abilities were not explicitly programmed.

Examples

- Better reasoning
- Code generation
- Few-shot learning
- Chain-of-thought reasoning
- Complex problem solving

These abilities often emerge only after the model reaches a certain scale.

---

8. Compute Optimal Training

Definition

Compute Optimal Training means choosing the best combination of:

- Model size
- Dataset size
- Compute budget

to achieve the highest performance for the available resources.

Goal

Train the most capable model **without wasting computation**.

Module 12: Fine-Tuning

1. Fine-Tuning

Introduction

A pretrained LLM has general language knowledge. **Fine-Tuning** further trains the model on a specific dataset so it performs better on a particular task or domain.

Example

A general LLM can be fine-tuned for:

- Medical AI
- Legal AI
- Coding Assistant
- Customer Support

---

2. Why Fine-Tune?

Fine-tuning helps an LLM:

- Improve task-specific performance
- Learn domain-specific knowledge
- Follow instructions better
- Adapt to company data
- Produce more accurate responses

---

3. Full Fine-Tuning

Definition

Full Fine-Tuning updates **all model parameters** during training.

Advantages

- Highest performance
- Learns task deeply

Disadvantages

- Very expensive
- Requires powerful GPUs
- High memory usage

---

4. Instruction Tuning

Definition

Instruction Tuning trains an LLM to **follow human instructions** using instruction-response datasets.

Example

Instruction

Summarize this article.

Output

A short summary of the article.

Benefit

Makes the model more helpful and conversational.

---

5. Domain Adaptation

Definition

Domain Adaptation fine-tunes an LLM on data from a **specific field or industry**.

Examples

- Healthcare
- Finance
- Law
- Education

Benefit

Improves accuracy on specialized topics.

---

6. Continual Learning

Definition

Continual Learning allows a model to learn **new knowledge over time** without starting training from scratch.

Benefits

- Learns new information
- Adapts to changing domains
- Saves training time

---

7. Catastrophic Forgetting

Definition

Catastrophic Forgetting occurs when a model **forgets previously learned knowledge** while learning new information.

Example

A medical model fine-tuned only on legal data may lose some medical knowledge.

Solution

- PEFT methods
- Replay training
- Regularization techniques

---

8. Parameter Efficient Fine-Tuning (PEFT)

Definition

PEFT updates **only a small subset of parameters** instead of the entire model.

Advantages

- Lower GPU memory
- Faster training
- Lower storage cost
- Maintains most pretrained knowledge

---

9. PEFT

Definition

PEFT (Parameter Efficient Fine-Tuning) is a collection of techniques for efficiently adapting large language models.

Popular PEFT Methods

- LoRA
- QLoRA
- Prefix Tuning
- Prompt Tuning
- P-Tuning
- Adapters

---

10. LoRA (Low-Rank Adaptation)

Definition

LoRA freezes the original model and trains only **small low-rank matrices** added to certain layers.

Advantages

- Very memory efficient
- Fast training
- Small storage requirements

Most Popular PEFT Method

LoRA is one of the most widely used fine-tuning techniques.

---

11. QLoRA

Definition

QLoRA combines **model quantization** with LoRA.

The base model is stored in lower precision (e.g., 4-bit), while LoRA layers are trained.

Advantages

- Very low memory usage

- Enables fine-tuning on smaller GPUs
- Performance close to full fine-tuning

---

12. Prefix Tuning

Definition

Prefix Tuning learns a small set of **trainable prefix vectors** that are added before the input sequence.

The original model remains frozen.

Advantages

- Efficient
- Low memory
- Task-specific adaptation

---

13. Prompt Tuning

Definition

Prompt Tuning learns **soft prompts (trainable embeddings)** instead of changing model parameters.

Difference

- Text Prompt → Written by humans
- Soft Prompt → Learned during training

---

14. P-Tuning

Definition

P-Tuning is an improved version of Prompt Tuning that uses trainable prompt embeddings with additional neural layers.

Advantages

- Better performance

- Efficient training
- Works well on many NLP tasks

---

15. Adapters

Definition

Adapters are **small neural network modules** inserted into Transformer layers.

Only the adapters are trained while the original model remains unchanged.

Advantages

- Modular
- Memory efficient
- Easy to switch between tasks

Module 13: Prompt Engineering

1. Prompt Basics

Introduction

Prompt Engineering is the practice of designing effective prompts to guide an LLM toward producing accurate, relevant, and useful responses.

A well-designed prompt often leads to significantly better results.

---

2. Prompt

Definition

A **Prompt** is the text or instruction given to an LLM to perform a task.

Example

Explain what Machine Learning is.

---

3. Context

Definition

Context provides background information that helps the model understand the task better.

Example

You are teaching beginner students.
Explain Machine Learning in simple language.

---

4. Instruction

Definition

An **Instruction** clearly tells the model what to do.

Example

Summarize the following article in five bullet points.

---

5. Input

Definition

The **Input** is the data or content that the model should process.

Example

Article:
Artificial Intelligence is transforming healthcare...

---

6. Output

Definition

The **Output** is the response generated by the model.

Example

• AI improves diagnosis.
• AI automates repetitive tasks.

---

7. System Prompt

Definition

A **System Prompt** sets the model's overall behavior, personality, or rules before the conversation begins.

Example

You are an expert Python teacher.
Always explain concepts in simple language.

---

8. User Prompt

Definition

A **User Prompt** is the request or question entered by the user.

Example

Explain Python loops with examples.

---

9. Prompt Techniques

Common prompt engineering techniques include:

- Zero Shot
- One Shot
- Few Shot
- Chain of Thought (CoT)
- Self Consistency
- Tree of Thoughts
- ReAct
- Step-Back Prompting
- Role Prompting

---

10. Zero Shot

Definition

The model performs a task **without any examples**.

Example

Translate:
Hello

---

11. One Shot

Definition

The prompt contains **one example** before the actual task.

Example

Example:
Good → Positive

Now classify:
Amazing

---

12. Few Shot

Definition

The prompt provides **multiple examples** to help the model understand the desired pattern.

Example

Happy → Positive
Sad → Negative
Excellent → Positive

Classify:
Terrible

---

13. Chain of Thought (CoT)

Definition

Chain of Thought encourages the model to reason through a problem step by step before giving the final answer.

Benefit

- Better reasoning
- Improved performance on complex tasks

---

14. Self Consistency

Definition

Self Consistency generates **multiple reasoning paths** and selects the most consistent final answer.

Benefit

Improves reasoning accuracy.

---

15. Tree of Thoughts (ToT)

Definition

Tree of Thoughts explores **multiple reasoning branches** before selecting the best solution.

Benefit

Useful for complex planning and decision-making tasks.

---

16. ReAct (Reason + Act)

Definition

ReAct combines **reasoning** with **actions**, such as using tools or performing intermediate steps, before producing the final answer.

Example

The model may:

1. Think about the problem.
2. Perform an action (e.g., use a calculator or search tool if available).
3. Continue reasoning.
4. Produce the final answer.

Note: Here, ReAct refers to the reasoning-and-action framework **without Retrieval-Augmented Generation (RAG)**.

---

17. Step-Back Prompting

Definition

Step-Back Prompting asks the model to first think about the broader concept before solving the specific problem.

Benefit

Improves reasoning and reduces mistakes on complex tasks.

---

18. Role Prompting

Definition

Role Prompting assigns a specific role to the model.

Example

You are a senior software engineer.
Explain recursion to a beginner.

---

19. Delimiters

Definition

Delimiters clearly separate different parts of a prompt.

Common Delimiters

- Triple quotes (""")
- Triple backticks (```)
- XML tags
- Markdown headings

Example

Summarize the following text:

"""
Artificial Intelligence...

"""

---

20. Structured Prompts

Definition

Structured prompts organize information into clear sections.

Example

Task:
Summarize

Input:
Article...

Output Format:
3 bullet points

---

21. Prompt Design

Definition

Prompt Design is the process of creating prompts that are clear, specific, and easy for the model to understand.

Good Prompt Characteristics

- Clear
- Specific
- Unambiguous
- Well-structured

---

22. Prompt Templates

Definition

A Prompt Template is a reusable prompt structure where variable values can be replaced.

Example

Explain {topic} for a {audience} in {style}.

---

23. Constraints

Definition

Constraints define rules that the model must follow while generating the output.

Examples

- Maximum 100 words
- Use simple English
- Return only JSON
- Do not use bullet points

---

24. Output Formatting

Definition

Output Formatting specifies how the response should be presented.

Examples

- Bullet points
- Table
- JSON
- Markdown
- HTML
- CSV

---

25. Prompt Iteration

Definition

Prompt Iteration is the process of refining a prompt through multiple revisions to improve the quality of the model's output.

Steps

1. Write an initial prompt.
2. Review the response.
3. Modify the prompt for clarity or specificity.

4. Repeat until the desired output is achieved.

Module 16: Reasoning in LLMs

1. Reasoning

Introduction

Reasoning is the ability of an LLM to analyze information, connect ideas, and solve problems by following logical steps rather than simply recalling memorized text.

Reasoning helps LLMs answer complex questions and make informed decisions.

---

2. Logical Reasoning

Definition

Logical Reasoning is the ability to draw valid conclusions using logical rules.

Example

All birds have wings.
A sparrow is a bird.

Conclusion:

A sparrow has wings.

Applications

- Question answering
- Decision making
- Problem solving

---

3. Mathematical Reasoning

Definition

Mathematical Reasoning is the ability to solve mathematical problems using calculations and logical steps.

Example

$12 \times 5 = 60$

Applications

- Arithmetic
- Algebra
- Word problems
- Quantitative reasoning

---

4. Multi-Step Reasoning

Definition

Multi-Step Reasoning solves problems by breaking them into several smaller steps.

Example

Question:
John has 5 apples.
He buys 3 more.
Then gives away 2.
How many apples remain?

Steps

- $5 + 3 = 8$
- $8 - 2 = 6$

Answer

6 apples

---

5. Commonsense Reasoning

Definition

Commonsense Reasoning uses everyday human knowledge to answer questions.

Example

Can an elephant fit inside a backpack?

Answer

No.

The model uses common world knowledge rather than memorized facts alone.

---

6. Symbolic Reasoning

Definition

Symbolic Reasoning manipulates symbols, rules, or formal logic instead of relying only on language patterns.

Examples

- Logic puzzles
- Mathematical expressions
- Programming code
- Rule-based systems

---

7. Planning

Definition

Planning is the ability to create an ordered sequence of actions to achieve a goal.

Example

Goal

Build a website

Plan

1. Gather requirements
2. Design UI
3. Develop frontend
4. Develop backend
5. Test
6. Deploy

---

8. Tool Reasoning (Concept Only)

Definition

Tool Reasoning is the ability to decide **when and how** to use external tools while solving a problem.

Examples of tools include:

- Calculator
- Search engine
- Database
- Code interpreter

Purpose

Choose the right tool when language knowledge alone is not enough.

---

9. Reflection

Definition

Reflection is the process where an LLM reviews its own reasoning or answer to identify possible mistakes.

Benefits

- Improves answer quality
- Detects errors
- Produces more reliable responses

---

10. Self Verification

Definition

Self Verification means checking whether the generated answer is consistent and logically correct before presenting it.

Example

The model:

- Solves a math problem
- Rechecks the calculation

- Returns the verified answer

Module 16 Summary

- **Reasoning** enables LLMs to solve problems beyond simple text prediction.
- Types include **Logical**, **Mathematical**, **Multi-Step**, **Commonsense**, and **Symbolic Reasoning**.
- **Planning** helps break complex goals into ordered actions.
- **Tool Reasoning** focuses on deciding when external tools are needed.
- **Reflection** and **Self Verification** improve answer quality by reviewing and checking outputs before returning them.

Module 17: Hallucination and Reliability

1. Hallucinations

Introduction

Although LLMs are powerful, they can sometimes generate information that is **incorrect, misleading, or completely fabricated**. These incorrect outputs are called **hallucinations**.

---

2. What is Hallucination?

Definition

A **Hallucination** occurs when an LLM generates information that appears believable but is false, unsupported, or invented.

Example

Question

Who invented the Internet in 1890?

The model might confidently generate a fictional answer, even though the premise is false.

---

3. Types of Hallucinations

1. Factual Hallucination

Produces incorrect facts.

Example

The Earth has two moons.

---

2. Citation Hallucination

Invents books, papers, authors, or references that do not exist.

---

3. Logical Hallucination

Produces reasoning that is internally inconsistent or invalid.

---

4. Instruction Hallucination

Ignores the user's instructions or adds unnecessary information.

---

4. Causes

Common causes of hallucinations include:

- Incomplete training data
- Ambiguous prompts
- Lack of real-time knowledge
- Weak reasoning
- Overgeneralization
- Predicting likely text instead of verified facts

---

5. Detection

Definition

Hallucination Detection is the process of identifying incorrect or fabricated responses.

Methods

- Fact checking
- Human review
- External knowledge sources
- Consistency checks
- Self verification

---

6. Mitigation

Definition

Mitigation refers to techniques that reduce hallucinations.

Common Methods

- Better prompts
- High-quality training data
- Fine-tuning
- Tool usage
- Self verification
- Human feedback

---

7. Reliability

Definition

Reliability is the ability of an LLM to consistently produce accurate and dependable responses.

A Reliable Model Should Be

- Accurate
- Consistent
- Stable
- Trustworthy

---

8. Calibration

Definition

Calibration means the model's confidence should closely match its actual correctness.

Example

If the model says it is **95% confident**, the answer should usually be correct.

---

9. Confidence

Definition

Confidence indicates how certain the model is about its prediction.

Note

High confidence does **not** always mean the answer is correct.

---

10. Truthfulness

Definition

Truthfulness measures whether the model's output is factually correct.

Goal

Generate responses based on accurate information instead of plausible-sounding false statements.

---

11. Faithfulness

Definition

Faithfulness measures whether the generated response stays consistent with the provided input or source information.

Example

If summarizing a document, the summary should not introduce new facts that were not present in the original text.

---

12. Robustness

Definition

Robustness is the ability of a model to produce reliable outputs even when inputs vary slightly.

Example

Different wording of the same question should produce similar correct answers.

---

13. Bias

Definition

Bias occurs when the model systematically favors or disadvantages certain groups, viewpoints, or outcomes.

Goal

Reduce unfair or skewed behavior during training and deployment.

---

14. Toxicity

Definition

Toxicity refers to harmful, offensive, abusive, or inappropriate generated content.

Goal

Minimize harmful outputs while maintaining helpful responses.

---

15. Fairness

Definition

Fairness means treating different people, groups, and perspectives equitably.

Goal

Produce balanced responses without unfair discrimination.

Module 18: LLM Evaluation

1. Evaluation

Introduction

LLM Evaluation measures how well a language model performs on different tasks.

Evaluation helps compare models, identify weaknesses, and improve future versions.

---

2. Perplexity

Definition

Perplexity measures how well a language model predicts the next token.

Interpretation

- Lower Perplexity → Better predictions
- Higher Perplexity → Poorer predictions

Common Use

Evaluate language modeling performance during pretraining.

---

3. BLEU

Full Form

Bilingual Evaluation Understudy

Definition

BLEU measures how closely generated text matches one or more reference texts.

Common Use

- Machine Translation

Higher BLEU indicates a closer match to the reference.

---

4. ROUGE

Full Form

Recall-Oriented Understudy for Gisting Evaluation

Definition

ROUGE measures the overlap between generated text and reference summaries.

Common Use

- Text Summarization

Higher ROUGE generally indicates better summary quality.

---

5. METEOR

Definition

METEOR evaluates generated text by considering:

- Exact word matches
- Synonyms
- Word stems
- Word order

Advantage

Often aligns better with human judgment than BLEU for some tasks.

---

6. BERTScore

Definition

BERTScore measures similarity using contextual embeddings instead of exact word matching.

Advantage

Captures semantic similarity even when different words are used.

---

7. Exact Match (EM)

Definition

Exact Match checks whether the generated answer exactly matches the correct answer.

Common Use

- Question Answering
- Reading Comprehension

---

8. Pass@k

Definition

Pass@k measures whether **at least one** of the model's **k generated outputs** is correct.

Common Use

- Code generation

Higher Pass@k means a greater chance of producing a correct solution.

---

9. Human Evaluation

Definition

Humans manually evaluate model outputs based on quality and usefulness.

Common Criteria

- Accuracy
- Fluency
- Relevance
- Helpfulness
- Safety

Human evaluation remains one of the most important evaluation methods.

---

10. Benchmarks

Definition

Benchmarks are standardized datasets used to compare LLM performance.

Popular benchmarks include:

- MMLU
- HELM
- BIG-bench
- HellaSwag
- GSM8K
- HumanEval
- TruthfulQA

---

11. MMLU

Full Form

Massive Multitask Language Understanding

Purpose

Evaluates knowledge and reasoning across many academic subjects.

---

12. HELM

Full Form

Holistic Evaluation of Language Models

Purpose

Evaluates models across multiple dimensions such as:

- Accuracy
- Fairness
- Robustness
- Efficiency

---

13. BIG-bench

Full Form

Beyond the Imitation Game Benchmark

Purpose

Tests advanced reasoning, language understanding, and emerging abilities across a wide variety of tasks.

---

14. HellaSwag

Definition

HellaSwag evaluates **commonsense reasoning** by asking the model to choose the most plausible continuation of a situation.

---

15. GSM8K

Full Form

Grade School Math 8K

Purpose

Evaluates mathematical reasoning using grade-school word problems.

---

16. HumanEval

Definition

HumanEval measures a model's ability to generate correct programming code.

Common Metric

- Pass@k

---

17. TruthfulQA

Definition

TruthfulQA evaluates whether a model gives **truthful answers** instead of common misconceptions or misleading responses.

Focus

- Truthfulness
- Factual accuracy
- Resistance to misinformation

Module 19: Efficient LLMs

1. Efficient LLMs

Introduction

As LLMs grow larger, they require more memory, computation, and inference time.

Efficient LLM techniques reduce resource usage while maintaining good performance.

---

2. Optimization

Definition

Optimization techniques improve the speed, memory usage, and efficiency of LLM training and inference.

Goals

- Faster inference
- Lower memory usage
- Lower cost
- Better scalability

---

3. Quantization

Definition

Quantization reduces the precision of model weights (e.g., from 32-bit to 16-bit, 8-bit, or 4-bit).

Example

FP32 → FP16 → INT8 → INT4

Advantages

- Smaller model size
- Faster inference
- Lower memory usage

Limitation

Very aggressive quantization may slightly reduce accuracy.

---

4. Pruning

Definition

Pruning removes unimportant weights or neurons from the model.

Benefits

- Smaller model
- Faster inference
- Lower memory usage

Limitation

Too much pruning can reduce model performance.

---

5. Knowledge Distillation

Definition

Knowledge Distillation trains a **small model (Student)** to imitate a **large model (Teacher)**.

Process

Large Teacher Model
↓
Generates Knowledge
↓

Small Student Model Learns

Advantages

- Faster inference
- Smaller model
- Lower deployment cost

---

6. Speculative Decoding

Definition

Speculative Decoding speeds up text generation by using a **small draft model** to predict candidate tokens, which are then verified by the larger model.

Benefits

- Faster generation
- Maintains similar output quality
- Reduces latency

---

7. Flash Attention

Definition

Flash Attention is an optimized attention algorithm that reduces memory usage and speeds up attention computation.

Advantages

- Faster attention computation
- Lower GPU memory usage
- Better training efficiency

Common Use

Widely used in modern Transformer implementations.

---

8. Paged Attention

Definition

Paged Attention manages the model's key-value (KV) cache efficiently by storing it in memory pages.

Benefits

- Efficient memory management
- Supports many concurrent users
- Faster inference for serving LLMs

---

9. Continuous Batching

Definition

Continuous Batching dynamically adds new requests into running inference batches instead of waiting for the current batch to finish.

Advantages

- Better GPU utilization
- Higher throughput
- Lower response latency

Module 20: Long Context Models

1. Long Context

Introduction

A **Long Context Model** can process and understand much longer input sequences than standard LLMs.

This allows the model to analyze large documents, books, conversations, or codebases.

---

2. Context Window

Definition

The **Context Window** is the maximum number of tokens an LLM can process at one time.

Example

If a model has a **128K context window**, it can process approximately **128,000 tokens** in a single request.

Larger Context Windows Help With

- Long conversations
- Large documents
- Code repositories
- Research papers

---

3. Sliding Window Attention

Definition

Sliding Window Attention allows each token to attend only to nearby tokens within a fixed window instead of the entire sequence.

Example

Window Size = **3**

A B C D E F G

Token **D** attends to:

B C D E F

Advantages

- Lower memory usage
- Faster computation
- Better scalability for long inputs

---

4. Memory Mechanisms

Definition

Memory Mechanisms help the model retain and reuse important information across long sequences.

Benefits

- Better long-term understanding
- Improved consistency
- More coherent long conversations

---

5. Chunking (Concept)

Definition

Chunking divides a long document into **smaller sections (chunks)** before processing.

Example

A 500-page book can be split into many smaller chunks for easier processing.

Benefits

- Handles long documents efficiently
- Reduces memory requirements
- Simplifies processing

---

6. RoPE Scaling

Definition

RoPE Scaling extends the usable context length of models that use **Rotary Positional Embeddings (RoPE)**.

Benefits

- Supports longer contexts
- Preserves positional information
- Improves long-range understanding

---

7. Position Interpolation

Definition

Position Interpolation adjusts positional embeddings so a model trained on shorter sequences can work with longer sequences.

Benefits

- Extends context length
- Avoids retraining from scratch
- Maintains model performance

---

8. Attention Sinks

Definition

Attention Sinks are special tokens or positions that consistently receive high attention and help stabilize attention patterns in long-context models.

Benefits

- Improved stability
- Better handling of very long contexts
- More reliable attention behavior

Module 21: Multimodal LLMs

1. Multimodal

Introduction

A **Multimodal LLM** can understand and generate information from **multiple types of data (modalities)**, not just text.

Common Modalities

- Text
- Images
- Audio
- Video

---

2. Vision Language Models (VLMs)

Definition

Vision Language Models combine **computer vision** and **language understanding** to process both images and text.

Applications

- Image captioning
- Visual question answering
- Image understanding
- Document analysis

---

3. Image Encoder

Definition

An **Image Encoder** converts an image into numerical feature representations (embeddings) that the LLM can understand.

Process

Image
↓
Image Encoder
↓
Image Embeddings
↓
LLM

---

4. Cross Attention

Definition

Cross Attention allows the language model to attend to information from another modality, such as image features.

Example

Image + Question

What is the person holding?

The LLM uses image features to answer the question.

---

5. Vision Transformer (ViT)

Definition

Vision Transformer (ViT) applies the Transformer architecture to images.

How It Works

- Splits an image into small patches.
- Treats each patch like a token.
- Processes the patches using Transformer layers.

Benefits

- Strong image understanding
- Scalable architecture

---

6. CLIP

Full Form

Contrastive Language-Image Pretraining

Definition

CLIP learns a shared representation for images and text by training them together.

Benefits

- Connects images with text
- Enables image search
- Supports zero-shot image classification

---

7. Image Understanding

Definition

Image Understanding is the ability of an LLM to analyze and interpret visual content.

Examples

- Object detection (concept)
- Image captioning
- Scene understanding
- Visual question answering

---

8. Image Generation (Concept)

Definition

Image Generation is the process of creating images from text descriptions.

Example

Prompt

A futuristic city at sunset.

Output

An AI-generated image matching the description.

Note: Image generation is typically performed by diffusion or other generative image models, while multimodal LLMs may coordinate or guide the process.

---

9. Audio Language Models

Definition

Audio Language Models process and generate audio-related information.

Applications

- Speech recognition
- Speech understanding
- Speech translation
- Voice assistants

---

10. Video Language Models

Definition

Video Language Models understand both visual frames and their temporal relationships in videos.

Applications

- Video summarization
- Video captioning
- Video question answering
- Action recognition